

AssetHub

Smart Asset Management & Resource Allocation Platform

Design Document — Open Projects 2026

Cultural Council, IIT Roorkee

1. Problem Understanding

The Cultural Council of IIT Roorkee manages a large shared pool of expensive equipment — DSLRs, studio lighting, PA systems, costumes, and stage props — across numerous events. Without a centralized system, coordination through WhatsApp, spreadsheets, and registers causes scheduling conflicts, lost equipment, and zero visibility.

Pain Points

Problem	Description
No Inventory Visibility	Admins cannot see what's available, out, or who has it
Fragmented Requests	Booking requests arrive via chats, emails, and verbal asks
Double-Booking Conflicts	Same equipment committed to two events on the same date
No Return Accountability	Assets go out but return dates are never tracked or enforced
Zero Analytics	No data on which equipment is overused, underused, or lost

Stakeholders

Role	Responsibilities	Key Needs
Admin	Manage inventory, approve/reject bookings, issue/return assets, view analytics	Dashboard, fast approvals, inventory accuracy
User (Student)	Browse assets, submit requests, track status, view history	Easy booking, status updates, transparency

Solution Scope

- Centralized inventory with category, condition, and quantity tracking
- Date-range booking with multi-unit conflict detection
- Multi-step admin workflow: PENDING → APPROVED → ISSUED → RETURNED
- Analytics dashboard (bar, pie, line charts) + append-only audit log
- Deployable on any server with a single Docker Compose command

2. System Architecture

Three-tier architecture: React SPA frontend → stateless Express.js REST API → SQLite via Prisma ORM. JWT tokens authenticate every request, eliminating server-side session state.

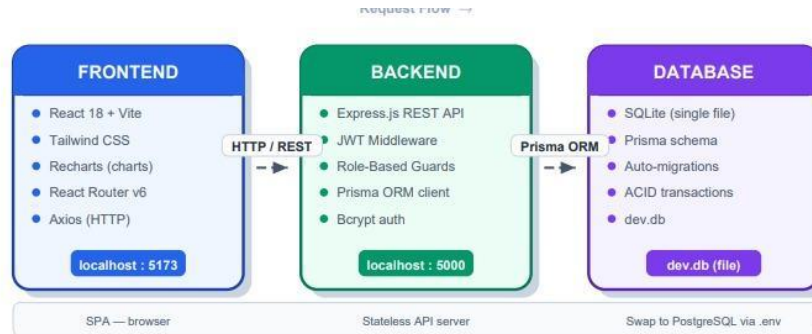


Figure 1 — Three-tier architecture with port mapping

Layer	Technology	Justification
Frontend	React 18 + Vite + Tailwind CSS	Fast HMR; utility-first CSS; component model fits dashboard UI
Routing	React Router v6	Nested routes; protected route wrappers for auth guards
Charts	Recharts	React-native; responsive by default; composable API
HTTP	Axios with interceptors	Auto-attaches JWT; global 401 handler redirects to login
Backend	Node.js + Express.js	Lightweight; JSON-native; large ecosystem; easy to deploy
Auth	JWT + bcryptjs	Stateless; bcrypt cost factor 12 for password hashing
ORM	Prisma v5	Type-safe queries; auto-migration; Prisma Studio for inspection
Database	SQLite	Zero-config single file; swap to PostgreSQL via one .env line
Deploy	Docker Compose	Nginx frontend + Express backend; one-command startup

Request Lifecycle

- Browser sends Authorization: Bearer <token> header via Axios
- authenticate middleware verifies JWT, attaches decoded user to req.user
- requireAdmin middleware checks req.user.role for admin-only routes
- Controller runs business logic via Prisma and returns JSON
- Sensitive mutations (approve + qty decrement) wrapped in Prisma.\$transaction()

3. Database Schema

Four entities. A User creates many Bookings; an Asset appears in many Bookings; both are referenced in AuditLog entries. Foreign keys enforced by Prisma ORM; cascading deletes handled in controllers.

USER & ASSET

Entity	Field	Type / Constraint	Description
USER	id / name / email	PK cuid / NOT NULL / UNIQUE	Auto ID; full name; login credential
USER	password / role	NOT NULL / DEFAULT 'USER'	bcrypt hash (cost 12); ADMIN or USER — first registrant = ADMIN
ASSET	id / name / category	PK cuid / NOT NULL / NOT NULL	Unique ID; display name; equipment type
ASSET	total_qty / available_qty	Default 1 / Default 1	Total units owned / units not currently allocated
ASSET	status / condition	Default ACTIVE / Default GOOD	ACTIVE MAINTENANCE RETIRED · EXCELLENT GOOD FAIR POOR

BOOKING

Field	Type / Constraint	Description
id / user_id / asset_id	PK cuid / FK User / FK Asset	Unique ID; who booked; which asset
quantity / purpose	DEFAULT 1 / NOT NULL	Units requested; reason for booking
start_date / end_date	NOT NULL / NOT NULL	Booking window
status	DEFAULT PENDING	PENDING APPROVED REJECTED ISSUED RETURNED OVERDUE
admin_note / issued_at / returned_at	Optional x3	Admin message; physical handout & return timestamps

AUDITLOG

Field	Type / Constraint	Description
id / user_id / asset_id	PK cuid / FK Optional / FK Optional	Log ID; actor; asset involved
action / details	NOT NULL / NOT NULL	Event type (e.g. BOOKING_APPROVED); human-readable description
created_at	DEFAULT now()	Immutable — logs are append-only, never updated or deleted

4. Entity Relationship Diagram

Crow's foot notation — one-to-many relationships between entities.



Figure 2 — Entity Relationship Diagram (AssetHub database)

Key Constraints & Integrity Rules

Constraint	Description
Booking Overlap Check	Aggregates SUM of quantities for all APPROVED/ISSUED bookings in the requested date range; proceeds only if remaining qty is sufficient.
Atomic Approve/Return	Status update + availableQty change run in a Prisma transaction — both succeed or both roll back, preventing inventory drift.
Cascading Delete Guard	Assets with PENDING, APPROVED, or ISSUED bookings cannot be deleted; enforced in the controller before issuing DELETE.
Role Enforcement	requireAdmin middleware runs on every admin route before the controller — cannot be bypassed client-side.
Email Uniqueness	UNIQUE constraint at DB level provides a second layer beyond the application-level check.

5. API Overview

Authentication

Method	Endpoint	Auth	Description
POST	/auth/register	Public	Register user. First user auto-assigned ADMIN.
POST	/auth/login	Public	Login; returns JWT token + user object.
GET	/auth/me	User	Returns current user from JWT payload.

Assets

Method	Endpoint	Auth	Description
GET	/assets	User	List assets. Params: search, category, status, page, limit.
GET	/assets/:id	User	Single asset with active bookings included.
POST	/assets	Admin	Create asset: name, category, totalQuantity, status, condition.
PUT	/assets/:id	Admin	Update asset; recalculates availableQty if totalQuantity changes.
DELETE	/assets/:id	Admin	Delete asset. Blocked if active bookings exist.

Bookings

Method	Endpoint	Auth	Description
GET	/bookings	User/Admin	List bookings. Admin sees all; User sees own.
POST	/bookings	User	Create booking; validates date range and qty.
GET	/bookings/:id	User/Admin	Single booking. Users can only read their own.
PATCH	/bookings/:id/approve	Admin	Approve PENDING; decrements availableQty.
PATCH	/bookings/:id/reject	Admin	Reject PENDING; optional adminNote in body.
PATCH	/bookings/:id/issue	Admin	Mark APPROVED as ISSUED; sets issuedAt timestamp.
PATCH	/bookings/:id/return	Admin	Mark RETURNED; increments availableQty.
PATCH	/bookings/:id/cancel	User	Cancel own PENDING or APPROVED booking.
GET	/bookings/audit	Admin	Paginated audit log of all system events.

Analytics

Method	Endpoint	Auth	Description
GET	/analytics/dashboard	Admin	Summary stats, chart data, recent bookings, top assets.
GET	/analytics/utilization	Admin	Per-asset utilization rates and booking counts.

6. Key Design Decisions

Decision	Rationale
SQLite over PostgreSQL	Zero-config dev setup; single file (dev.db). Swap to PostgreSQL by changing one .env line — no application code changes needed.
Stateless JWT Auth	No session store; horizontally scalable. Tokens expire in 7 days. Axios 401 interceptor clears localStorage and redirects automatically.
Prisma Transactions	Approve and Return wrap booking status + availableQty in Prisma.\$transaction() — both succeed or both roll back, preventing inventory drift.
Overlap Detection	Backend sums quantities of APPROVED/ISSUED bookings in the requested date range; proceeds only if remaining qty is sufficient.
First-User Admin	First registrant auto-assigned ADMIN — no seed script needed. Works in fresh installs and demo scenarios.
Role Middleware	authenticate and requireAdmin are Express middleware, not inline controller logic — separated concerns, reusable, easy to extend.
Append-Only Audit Log	Every state change writes an AuditLog record that is never updated or deleted — full forensic trail for dispute resolution.

7. Feature Checklist

Status	Feature	Description
✓	User Authentication	Register, login, JWT sessions, role-based access (Admin/User)
✓	Inventory Management	Add, edit, delete, categorize assets; quantity management
✓	Asset Discovery & Booking	Browse/search/filter; date-range booking; quantity validation
✓	Approval Workflow	Admin approve/reject with notes; users see live status
✓	Issue & Return Tracking	Admin marks issued/returned; inventory auto-updates atomically
✓	Analytics Dashboard	Bar, pie, line charts; top assets; utilization rates; summary cards
✓	Borrowing History	Users see past bookings; admins see all system activity
★	Audit Logs	Append-only event log for every admin and user action
★	Asset Health Tracking	Condition field: EXCELLENT / GOOD / FAIR / POOR per asset
★	Dockerized Deployment	Docker Compose with Nginx frontend + Express backend
★	Demo Credentials	Quick-fill buttons on login page for instant demo access

8. Security & Frontend Architecture

Security Considerations

Area	Implementation
Password Hashing	bcrypt cost factor 12 — computationally expensive against brute-force attacks
JWT Expiry	7-day token; stored in localStorage; cleared automatically on 401 response
Input Validation	Server-side validation on all inputs; quantity cannot exceed available stock
Role Guards	Every admin endpoint has requireAdmin middleware; cannot be bypassed client-side
CORS Policy	Only localhost:5173 and localhost:3000 allowed; configurable via .env in production
SQL Injection	Prisma ORM uses parameterized queries; raw SQL never used in the codebase

Frontend Component Architecture

Layer	Components	Responsibility
Pages	Login, Register	Public routes — unauthenticated entry points
Pages	Dashboard	Admin home — summary stats, recent activity, chart widgets
Pages	Assets	Browse, search, filter inventory; open booking modal
Pages	Bookings	Admin view of all bookings with status filter and actions
Pages	MyBookings / History	User's active bookings and past borrowing history
Pages	AuditLogs	Paginated append-only event stream (admin only)
Layout	Sidebar + Layout	Role-aware navigation; wraps all authenticated pages
UI	StatusBadge, Spinner, BookingModal	Shared components used across pages

State & Data Flow

- AuthContext (React Context) — exposes user object, login(), logout(), and isAdmin helper to all components
- utils/api.js — single Axios instance; request interceptor attaches JWT, response interceptor handles 401
- No Redux/Zustand needed — server state fetched per-page with useEffect; auth state via context
- Protected route wrappers redirect unauthenticated users to /login and non-admins from admin pages